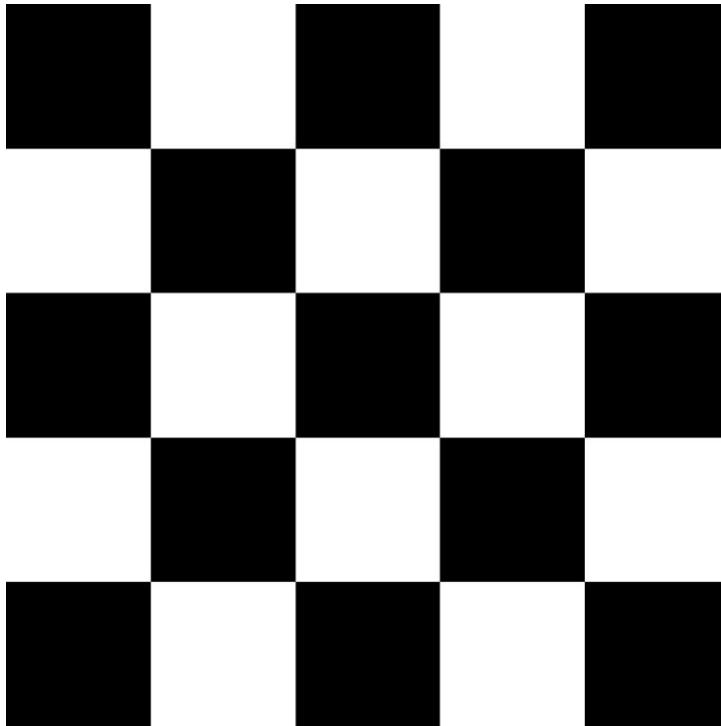


```
In21: import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Image
%matplotlib inline
```

```
[5]: Image(filename="checkerboard_18x18.png")
```



Out [5]:

```
In
# Reading image as gray scale.
cb_img = cv2.imread("checkerboard_18x18.png", cv2.IMREAD_GRAYSCALE)
# Print the image data (pixel values), element of a 2D numpy array.
# Each pixel value is 8-bits [0, 255]
print(cb_img)
cv2.imshow('Grayscale Image', cb_img)
```

```
[7]:
```

```
# Displaying the image attributes
print("Shape of the image: cb_img. shape)
print("Data type of the image: cb_img. dtype)
print("Size of the image: cb_img.size)
```

Shape of the image: (360, 360)

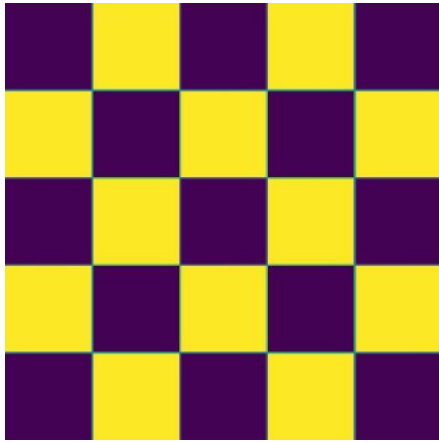
Data type of the image: uint8

Size of the image: 129600

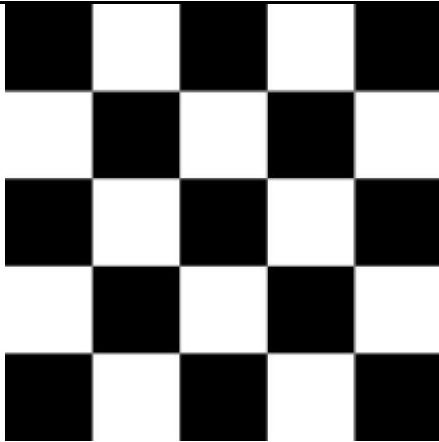
```
In
# Displaying image using Matplotlib.
```

```
plt.imshow(cb_img)
plt.title("Image Display Using Matplotlib")
plt.axis("off")
# To turn off axes
plt.show()
```

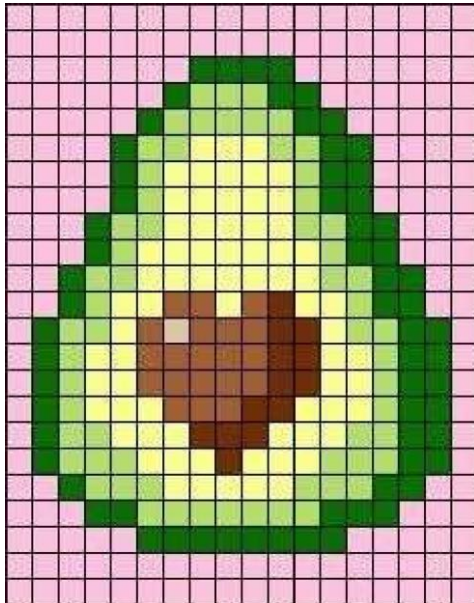
Image Display Using Matplotlib



```
[ In[9]: # Set color map to gray scale for proper rendering. plt.imshow(cb_img, cmap - " gray"
        ) PI t . axis ( "off" )
        PI t . show()
```



```
In [10]: # Read and display image Image( " avacado. jpg" )
```



Out[10]:

11]:

```
# Reading the image avacado_img = cv2.imread("avacado. jpg"
, cv2. IMREAD_COLOR)

# Printing the shape of the image print("Image shape (height, width,
channels) is : ", avacado_img. shape)
```

```
# Printing the size of the Image print("Image size is : 'I avacado_img.
size)

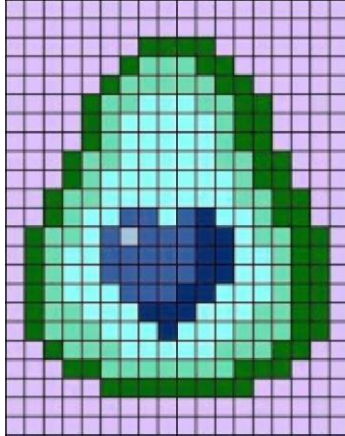
# Printing the data-type of the Image print("Data type of image is :
', avacado_img.dtype)
```

Image shape (height, width, channels) is : (300, 235, 3)

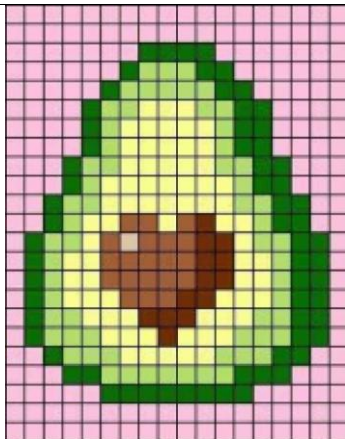
Image size is : 211500

Data type of image is : uint8

```
I n[12] : # Displaying the Image us ing matplotlib pit . imshow(avacado_img) pit
. axis ( "off" ) pit . Show()
```



```
n [13] : # Reversing the channels of the colon Image
          avacado_img_channels_reversed = avacado_img[:, :, ::-1]
I plt . imshow(avacado_img_channels_reversed)
imshow(      ) pit . axis ( "off" ) pit . show()
```



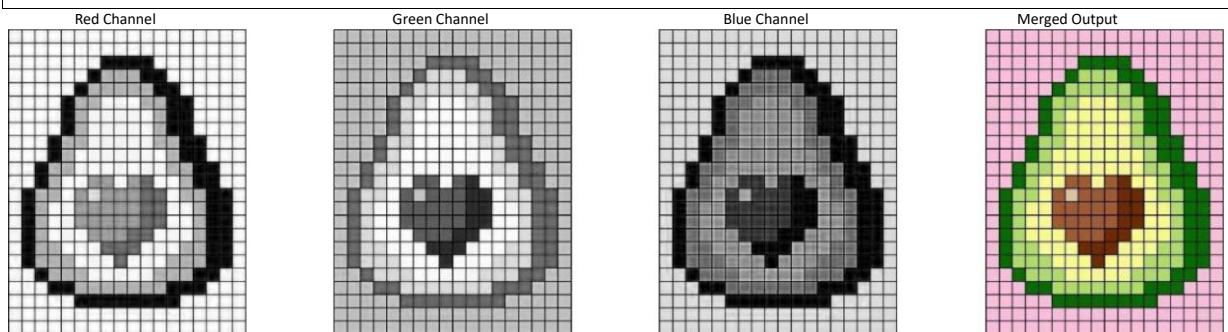
```
n [14] : # Split the Image into the B, G, R components b
          g, r cv2.sp1it(avacado_img)
I # Show the channels pit .
figure(figsize [20, 5] )
#      Red      Channel
plt.subplot(1, 4, 1)
plt.imshow(r, cmap "
gray") plt.tit1e( " Red
Channel") pit . axis ( "off 'l
")
```

```
# Green Channel
plt.subplot(l, 4, 2)
plt.imshow(g, cmap = "
gray plt.title("Green
Channel") plt . axis (
"off" )
# Blue Channel
plt.subplot(l, 4, 3)
plt.imshow(b, cmap = "
gray plt.title("Blue
Channel") plt . axis (
"off" )
# Merge the individual channels into
merged_img = cv2.merge((b, g, r) )

# Show the merged output
plt.subplot(l, 4, 4)

plt.title("Merged Output
plt . axis ( "off" )
plt . show()
```

a BGR image



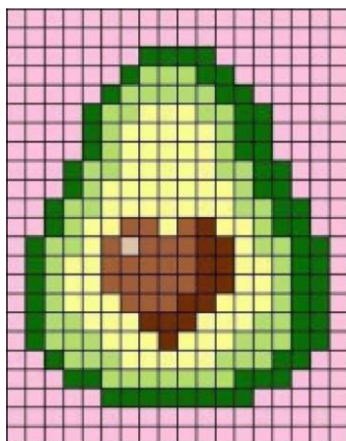
In

[15] :

```
# OpenCV stores color channels in a different order than most other applications (BGR)
avacado_img_bgr = cv2.imread("avacado.jpg", cv2.IMREAD_COLOR)

# Converting BGR to RGB avacado_img_rgb =
cv2.cvtColor(avacado_img_bgr, cv2.COLOR_BGR2RGB)
plt.imshow(avacado_img_rgb) plt.title("BGR to
RGB") plt . axis ( "off" )
plt . show()
```

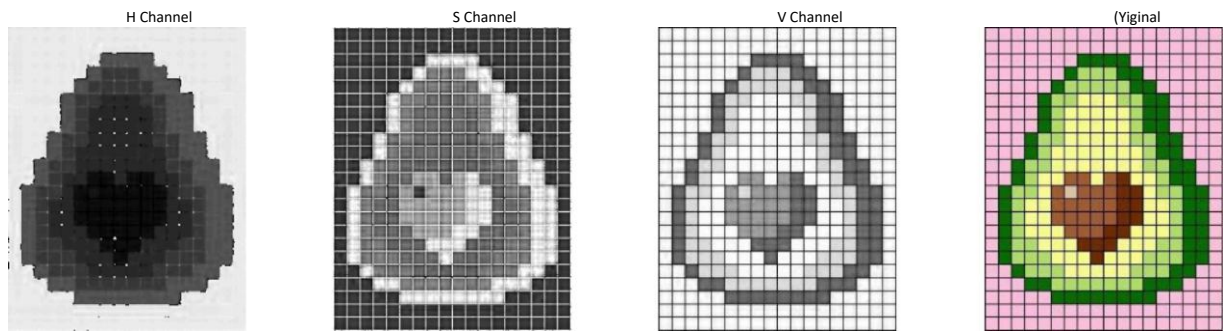
BGR to RGB



```
In [16]: # Converting to HSV avacado_img_hsv = cv2.COLOR_BGR2HSV)
cv2.cvtColor(avacado_img_bgr,

# Split the image into H, S, V components h, s,
v = cv2.split(avacado_img_hsv)

# Show the channels
plt.figure(figsize = [20,
5])
# Hue Channel
plt.subplot(1, 4, 1)
plt.imshow(h, cmap = "
gray" plt.title("H
Channel") plt.axis (
"off" )
# Saturation Channel
plt.subplot(1, 4, 2)
plt.imshow(s, cmap -
gray" plt.title("S
Channel") plt.axis (
"off" )
# Value Channel
plt.subplot(1, 4, 3)
plt.imshow(v, cmap - "
gray")
plt.title("V Channel")
plt.axis ( "off" )
# Show the original
image plt.subplot(1, 4,
4) plt .
.imshow(avacado_img_rgb)
plt.title( "Original " )
plt.axis ( "off" )
plt.show()
```



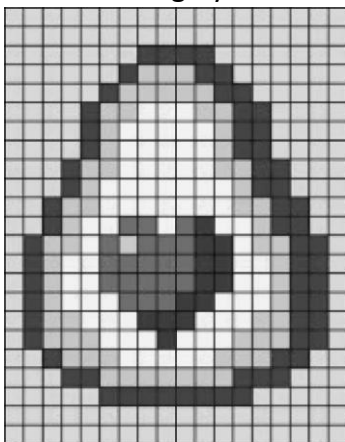
```

n[18]: # Converting BGR to grayscale
avacado_img_gray = cv2.cvtColor(avacado_img_bgr, cv2.COLOR_BGR2GRAY)

plt.
plt.title("BGR to grayscale")
plt.axis("off")
plt.show()

```

BGR to grayscale



In [19]

:

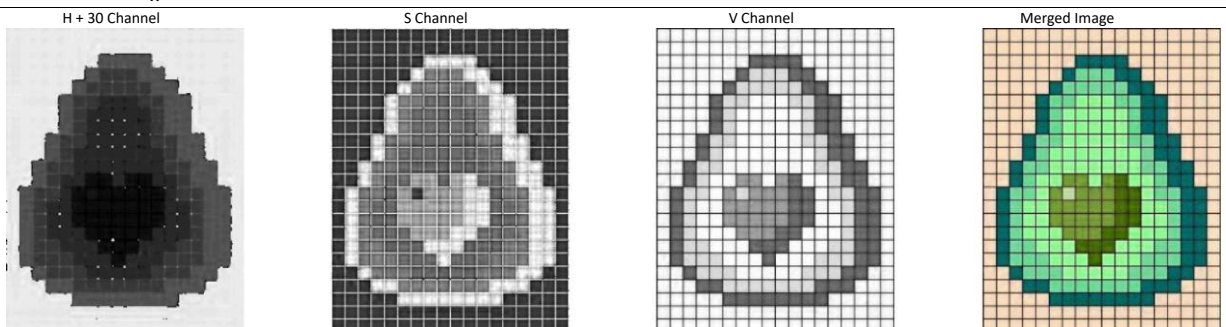
```

h_new = h + 30
avacado_img_merged = cv2.cvtColor(h_new, cv2.COLOR_HSV2RGB)
cv2.merge((h_new, s, v)) avacado_img_rgb =
cv2.cvtColor(avacado_img_merged,

# Split the image into the B, G, R components = cv2.
    split(avacado_img_merged),

# Show the channels
plt.figure(figsize = [20, 5])
# Hue Channel
plt.subplot(1, 4, 1)
plt.imshow(h, cmap = "
gray") plt.title("H + 30
Channel") plt.axis (
"off" )
# Saturation Channel
plt.subplot(1, 4, 2)
plt.imshow(s, cmap =
gray plt.title("S
Channel") plt.axis (
"off" )
# Value Channel plt.subplot
(1, 4, 3) plt.imshow(v, cmap =
" gray
plt.title("V Channel")
plt.axis ( "off" )
# Show the original image
plt.subplot(1, 4, 4)
plt.imshow(
Image")
plt.axis ( "off" )
plt.show()

```



In [20]

:

```

# Saving the image cv2. imwrite (
"avacado_saved . jpg" ,
avacado_img_rgb)

```

Out[20] : True

]: